

Week 13: Searchable Student Roster

Overview

This assignment turns Week 13 concepts into a small, finishable Java program or project milestone. Start from the provided template, preserve the public class filename, and run the program in IntelliJ before submitting.

Objectives

- Design a small immutable object with meaningful methods.
- Create and traverse an array of object references.
- Implement a sequential search that uses object equality.

Required reading - one new chapter

- [Think Java Chapter 12 - Arrays of Objects](#)

Optional supplemental reading - previously covered chapters

- [Chapter 7 - Arrays and References](#)
- [Chapter 11 - Designing Classes](#)

Supplemental videos

- [Arrays of objects](#)

Provided file input and output example

The starter already contains a complete `saveAndLoadRoster` method. It writes the finished `Student` array to `students.txt` and reads the file back. Read and run this method as a working example; the required student work focuses on immutable `Student` objects, an array of object references, traversal, equality, and sequential search.

Complete these steps

- Create `Week13ClassroomRoster` and a class named `ClassroomRoster`.
- Copy the starter into `ClassroomRoster.java`.
- Keep the `Student` fields private and final.
- Complete the `Student` constructor, `getId`, `getName`, `toString`, and `equals` methods.
- Make `toString` return the student's ID, a space-hyphen-space, and the student's name, such as `101 - Ada`.
- Make `equals` return true when two `Student` objects have the same ID and name.
- Create `Ada` with ID 101, `Grace` with ID 102, and `James` with ID 103 in the provided `Student` array.
- Complete `printRoster` with an enhanced for loop that prints every `Student` object.
- Complete `findStudent` with a standard for loop that returns the matching index or -1 when the target is absent.
- Read the completed `saveAndLoadRoster` method. It is a working file input/output example and should not need modification.
- Run the program and confirm the roster displays, `Grace` is found at index 1, `Linus` produces -1, and `students.txt` is written and loaded.
- Submit `ClassroomRoster.java` and the generated `students.txt` file.

What will be checked

Use this list as your final review before submitting.

- Student has private final ID and name fields plus completed constructor, getters, toString, and equals methods.
- Ada, Grace, and James are stored in a Student array and printed by traversing the array.
- findStudent uses sequential search and returns index 1 for Grace and -1 for Linus.
- The provided file example creates students.txt and loads its contents.
- The submitted file contains no unfinished TODO marker from the starter template.